

SKILLS

LANGUAGES	TypeScript, JavaScript, Python, Bash, SQL
AI & AGENTS	Agent orchestration, LLM evaluation, tool calling, context engineering, system-prompt design, structured output, MCP, Claude Code
WEB & BACKEND	React, Next.js, Node.js, REST APIs, PostgreSQL
TESTING	Playwright, Appium, Page Object Model, CI test infrastructure, fault injection
INFRASTRUCTURE	Docker, GitHub Actions, CI/CD, Git, Linux/WSL2

WORK EXPERIENCE

QA Wolf

QA Lead

10/2025 to Present

- **Lead QA for Harvey, an \$11B AI legal platform:** own test strategy and the client relationship, scope and outline new coverage with their CSMs and product stakeholders in recurring client meetings, and own the delivery reporting they see
- Maintain **1,600+ Playwright/TypeScript E2E tests** across 13 product domains on an **82-class Page Object framework (~76K lines)** with a compiler-enforced PageFactory registry; executed platform-wide POM codemod and environment migrations across the full suite
- Made failure evidence **machine-consumable for an AI agent:** run logs, Playwright traces, failure video and live application DOM pulled programmatically instead of screenshotted by a human, the integration that made autonomous triage possible
- Triage failures by root cause rather than one test at a time: a single call-graph census pinned a **suite-wide defect across 71 affected tests** in one pass, a re-run-first policy proved a third of batch failures were infrastructure contention needing no code change, and open bugs are **re-verified on command** by re-running the tests behind each report, never by editing it
- Converted **~1,300 client spec cases** into a machine-readable ground-truth dataset, then gated every test against its own spec steps before shipping, so coverage was verified rather than assumed

QA Wolf

QA Engineer

07/2025 to 10/2025

- **Promoted to QA Lead within three months of hire** on delivery metrics
- Validated non-deterministic LLM behavior across **65 tests:** agentic document workflows, streaming completions with multi-minute retry windows, citation integrity and knowledge-source attachment, spanning native PC, Word Web, iOS/Appium, email and Twilio SMS/MFA, and stealth-browser OAuth against anti-bot protections

Zeta Global

Software Developer

02/2023 to 07/2025

- Built internal **React/Next.js dashboards** with real-time PostgreSQL filtering, adopted across teams as their working view into the data and retiring a standing manual reporting job
- Built a **Python collection service** over REST APIs and SQL that consolidated scattered sources into one validated pipeline, replacing manual gathering and measurably improving the accuracy of the data teams reported on
- Owned the **deployment path end to end:** Git, GitHub Actions and Docker pipelines that tested and deployed both systems

PROJECTS

The Control Room

Human-in-the-Loop AI Automation

Personal project, 2026 to Present

kxvin1.github.io/control-room

- Cut a broken test from **ten manual hand-offs down to two:** an AI agent owns the repair, one always-on screen surfaces only the steps that still need a person, and the day's reading load drops from **thirty failures to two decisions**
- Designed evidence gates so the agent **cannot self-certify:** "ready" requires two clean local runs plus one infrastructure run against the same commit, run id attached for audit; proof runs go out **twenty at a time** on the vendor's infrastructure. **500+ automated self-checks** guard the harness itself, including fault-injection tests that must catch a planted failure before a checker is trusted
- Made the agent's work **observable:** every request, reply and model call is derived from the session transcripts into an inbox with a derived status, a per-day **cost ledger** that measures each optimisation before it is kept or dropped, and a retention layer that maps every large file to a rule
- Cut agent context cost from **3-8k tokens to ~1k** per investigation by replacing raw page snapshots with filtered evidence reports, and live-DOM inspection from **62s to 17s**, so long investigations stopped exhausting context
- Built a **measurable self-improvement loop:** every correction becomes a ledger row read into the next session, a written rule, and where possible a check that fails if it recurs; repeat corrections get their capture fixed, a weekly audit flags rules with no check, and the agent files its own bugs
- Architected panels and agent as **decoupled processes communicating only through plain-text files**, so either side restarts without losing state, and the session **hands off to itself:** a hand-off written from live state at each close, and a context verdict that calls a fresh session before the limit

EDUCATION

App Academy

1000-hour full stack web development, 07/2022

·

CSU Northridge

B.S. Psychology, 12/2019